Auto-Connect to Local AI CLIs: Implementation Guide

Nov 2, 2026 · @Liam Blackshaw-Brown

What this gives you

Your app uses the AI tools already installed and signed in on the user's computer, so the work runs on their Claude and ChatGPT subscriptions instead of API credits. Realm (github.com/31Carlton7/realm, MIT) does this in four moves, and this guide rebuilds each one in your own project.

1. **Find the tools.** On launch, borrow the PATH the user's Terminal sees, then look up each known CLI by name.
2. **Ask each tool if it is signed in.** Run the tool's own status command. Never read its token files.
3. **Drive each tool through its official app mode.** Claude Code through the Claude Agent SDK, Codex through `codex app-server`, and about ten others through one shared protocol called ACP.
4. **Offer Install and Sign-in buttons.** Run the vendor's own install and login commands for the user, showing each command before it runs.

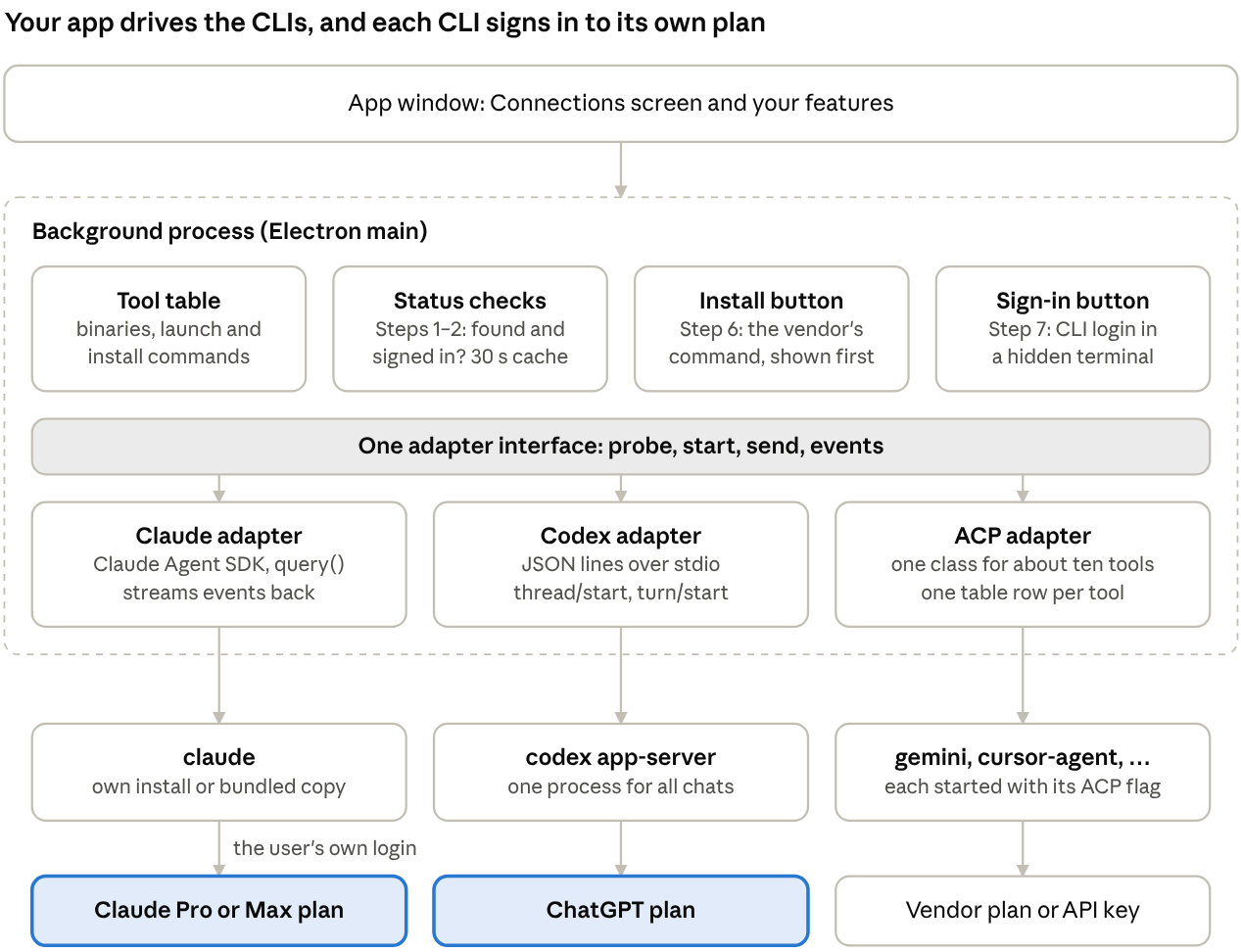
What you get: detection with no setup, Claude Code and Codex billed to the user's subscription, about ten more tools through one adapter, and one-click install and sign-in.

What you don't get:

- Gemini on a free personal login. Realm's code reports Google ended that tier for this use, so Gemini needs an API key or Vertex AI.
- Claude Code or Codex calling *your* app. That is a separate feature (an MCP server), and Realm hands its tools to each session rather than editing the user's settings.
- Permission to offer "Sign in with Claude" to other people. See Rules and risks before you publish.

Project layout

My recommendation: build it as an Electron app with a TypeScript background process. The Claude Agent SDK and `node-pty` are both Node libraries, which is why Realm made the same choice.



how the parts connect · app, background process, CLIs, plans

Only the adapters keep conversations open. Status checks, Install and Sign-in run the same CLIs briefly, and your app never holds a password or token.

```
my-app/
  src/main/                  Electron main process
    login-shell-path.ts      Step 1 PATH fix
    tools.ts                 the tool table: binary, launch, install, login
    probe.ts                 Steps 1-2 status checks, 30 s cache
    install.ts               Step 6 install runner
    signin.ts                Step 7 sign-in in a pseudo-terminal
    adapters/
      types.ts               the AgentAdapter interface
      claude.ts              Claude Agent SDK
      codex.ts               codex app-server
      acp.ts                 every ACP tool
      jsonrpc.ts             JSON-lines messaging
  src/renderer/              app window: Connections screen, your features
```

Tauri also works, but only with a Node sidecar for the Agent SDK, which is a second program to package.

Step 1 — Find the user's tools

Fix the PATH before anything else. A Mac app opened from Finder only sees `/usr/bin:/bin:/usr/sbin:/sbin`, so it cannot find anything installed with Homebrew, npm or uv.

1. **Borrow the Terminal's PATH at startup.** Run the user's login shell once, read the `PATH=` line between two markers, and put it in front of your own PATH. The markers matter because shell startup files print banners first.
2. **If that fails, add the usual folders:** `/opt/homebrew/bin`, `/usr/local/bin` and `~/.local/bin`. Skip all of this on Windows, where apps already get the user's PATH.
3. **Keep one table of known tools** (below). Every other part of the app reads binary names and launch commands from it.
4. **Look each binary up yourself.** Walk the PATH folders and check the file is executable. On Windows, try each PATHEXT ending, because npm installs `claude.cmd`, not `claude`.
5. **Count a tool as found only if it runs.** Call `<bin> --version` with a 5-second timeout. A leftover shim whose Node install is gone is a file that won't start.
6. **Let an environment variable override each path** (Realm uses `REALM_CLAUDE_BIN`, `REALM_CODEX_BIN` and so on). Tests and unusual installs both need it.

```
// Run once in the Electron main process, before spawning any CLI
const out = execFileSync(process.env.SHELL ?? "/bin/zsh",
  ["-l", "-i", "-c", "echo __S__; /usr/bin/env; echo __E__"],
  { timeout: 5000, encoding: "utf8" });
const block = out.slice(out.indexOf("__S__"), out.lastIndexOf("__E__"));
const login = block.split("\n").find(l => l.startsWith("PATH="))?.slice(5);
if (login) process.env.PATH = `${login}:${process.env.PATH}`;
```

Tool	Binary	How you drive it	Launch command
Claude Code	`claude`	Claude Agent SDK	handled by the SDK
Codex	`codex`	JSON-RPC app server	`codex app-server`
Gemini CLI	`gemini`	ACP	`gemini --acp`
Cursor	`cursor-agent`	ACP	`cursor-agent acp`
GitHub Copilot	`copilot`	ACP	`copilot --acp`
OpenCode	`opencode`	ACP	`opencode acp`
Qwen Code	`qwen`	ACP	`qwen --acp`
goose	`goose`	ACP	`goose acp`
Grok	`grok`	ACP	`grok agent stdio`
OpenHands	`openhands`	ACP	`openhands acp`
Hermes	`hermes`	ACP	`hermes acp`
fx	`fx`	ACP	`fx acp`

Realm confirmed each launch command against the live CLI in September 2026. CLIs rename flags, so keep them in this one table and re-check them when a tool updates.

Optional: ship Claude Code inside your app. The Claude Agent SDK includes a native `claude` binary in a per-platform package. Realm falls back to it when no `claude` is on PATH, so Claude works on a Mac with nothing installed.

Step 2 — Check each tool is signed in

Ask each CLI itself whether it is signed in, and keep "can't tell" as its own answer. Only a definite "no" should ever show a Sign-in prompt.

Tool	Command	Signed in	Signed out
Claude Code	`claude auth status --json`	`{"loggedIn": true}`	`{"loggedIn": false}` and exit code 1, so read stdout even when it "fails"
Codex	`codex login status`	exit code 0	exit code 1 with "Not logged in"; any other error means unknown
ACP tools	none reliable	unknown until a session starts	an auth error at `session/new`

- **Never read token files.** On a Mac, Claude Code keeps its login in the Keychain. Realm found `~/.claude/.credentials.json` five months stale while the real login was current.
- **"Signed in" is not "works".** Codex reported healthy logins whose tokens had been revoked. The real failure arrives at `thread/start` with `error.data.action === "relogin"`, so catch it there and say "Run `codex login`".
- **Return one shape for every tool**, so the UI never special-cases a CLI.
- **Run all checks in parallel and cache them for 30 seconds.** Each one starts a process. A "Check again" button skips the cache.
- **Several accounts per tool** are a different config folder per account: `CLAUDE_CONFIG_DIR` for Claude Code, `CODEX_HOME` for Codex.

```
type ProbeResult = {
  kind: string;              // "claude", "codex", "acp:gemini" ...
  available: boolean;        // --version ran
  version: string | null;
  loggedIn: boolean | null;  // null = can't tell, never treat as false
  reason: string | null;     // what to show the user, e.g. "run `codex
login`"
};
```

Step 3 — Run Claude Code

Use Anthropic's Claude Agent SDK (`@anthropic-ai/claude-agent-sdk`). It starts the `claude` program for you, so every request runs under the user's own Claude Code login.

- **`query({ prompt, options })`** returns a stream of messages: Claude's text, each tool it uses, and a final result.
- **`pathToClaudeCodeExecutable`** points at the binary Step 1 found. Leave it unset to use the copy bundled with the SDK.
- **`cwd`** is the folder the work happens in.
- **`permissionMode`** and **`canUseTool`** decide whether Claude asks before editing files or running commands. Send those questions to your UI.
- **`mcpServers`** hands the session your app's own tools, through an MCP server your app runs. This is how Realm adds features without touching `~/.claude`.
- **Remove `ANTHROPIC_API_KEY` from the child's environment** unless the user picked API billing. When that variable is set, Claude Code uses the key and bills API credits.
- **`settingSources: []`** shuts out the user's own `~/.claude` settings, skills and hooks. It also drops logins stored in `settings.json`, so Realm copies those back in through the environment (`settings-auth.ts`). Leave this option alone unless you need the isolation.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const { ANTHROPIC_API_KEY, ...env } = process.env;  // keep subscription
billing
for await (const msg of query({
  prompt: "Summarise the risks in ./notes.md",
  options: { cwd: projectDir, pathToClaudeCodeExecutable: claudeBin ??
undefined, env },
})) {
  if (msg.type === "assistant") render(msg);
  if (msg.type === "result") finish(msg);
}
```

For a quick question in and answer out, the same `query()` with a short prompt is enough. Realm writes its session titles and summaries this way.

Step 4 — Run Codex

Run `codex app-server`, Codex's official mode for other apps. Your app writes JSON lines to its input and reads JSON lines back, and one process can serve every Codex conversation.

1. **Start it** with input, output and error output all piped. Keep reading the error output, or it fills up and the process stalls.
2. **Send** **`initialize`** with your app's name and version, wait for the reply (time out after a few seconds), then send the `initialized` notice.
3. **Optional:** **`model/list`** returns the models the user's plan can use, for a model picker.
4. **`thread/start`** opens a conversation with a working folder, model and approval settings. A stale login fails here, with `relogin` in the error.
5. **`turn/start`** sends the user's message to that thread.
6. **Read the stream:** `item/agentMessage/delta` carries text, `item/started` and `item/completed` mark each step, and `turn/completed` ends the turn.
7. **Answer approval requests** Codex sends you: `item/commandExecution/requestApproval` and `item/fileChange/requestApproval`.
8. **`turn/interrupt`** stops a running turn, and `thread/resume` reopens an old conversation.

```
const cx = spawn(codexBin, ["app-server"], { stdio: ["pipe", "pipe", "pipe"]
});
cx.stderr.on("data", () => {});  // drain logs or it can stall
const send = (m: object) => cx.stdin.write(JSON.stringify({ jsonrpc: "2.0",
...m }) + "\n");
send({ id: 1, method: "initialize", params: {
  clientInfo: { name: "my-app", title: "My App", version: "0.1.0" },
  capabilities: { experimentalApi: true } } });
// on reply: send({ method: "initialized" }), then thread/start and
turn/start
```

Gotchas Realm hit (codex-cli 0.146 to 0.154):

- One JSON object per line, with no `Content-Length` headers.
- Replies leave out `"jsonrpc":"2.0"`, so don't reject them for it.
- Codex numbers its own requests from 0, which collides with your ids. Track your pending requests apart from the requests Codex sends you.
- Every error uses code -32600, so read the message and `data`, not the code.

- `codex app-server generate-ts` prints TypeScript types for the installed version.
- Broken MCP servers in the user's `~/.codex` config log "failed" but don't stop the session. Show them as warnings.

Step 5 — Run Gemini, Cursor, Copilot and the rest over ACP

Write one ACP client and every ACP row in the Step 1 table works through it. ACP (Agent Client Protocol) uses the same JSON-lines idea as Codex, with one shared set of messages across tools.

1. **Start the tool** with its launch command from the Step 1 table.
2. **`initialize`** with `protocolVersion: 1` and what your app can do, for example `clientCapabilities: { fs: { readTextFile: true, writeTextFile: true }, terminal: false }`.
3. **`session/new`** with the working folder and any MCP servers. Sign-in failures show up here, so show that tool's sign-in hint (below). The reply may list models under `configOptions` or `models`.
4. **`session/prompt`** sends the user's message.
5. **Read `session/update` notices** (text chunks, tool calls, plans) until the prompt request returns.
6. **Answer the tool's requests to you:** `session/request_permission` (pick an allow or deny option) and file reads such as `fs/read_text_file`.
7. **`session/cancel`** stops a turn, `session/load` resumes a conversation, and `session/set_model` or `session/set_mode` change settings.

ACP has no "list models" call. Realm opens a throwaway `session/new`, reads the models off the reply, and closes it.

Tool	How the user signs in
Cursor	`cursor-agent login`
GitHub Copilot	`copilot login`
OpenCode	`opencode auth login`
Grok	`grok login` (needs SuperGrok or X Premium), or `XAI_API_KEY`

Tool	How the user signs in
Qwen Code	run `qwen` once, or set `OPENAI_API_KEY`
fx	`fx login` (Vercel), or `AI_GATEWAY_API_KEY`
Hermes	`hermes model`, after installing its ACP extra
OpenHands	run `openhands` once and pick a model in `/settings`
goose	`goose configure`: pick a provider and enter its API key
Gemini CLI	Gemini API key, Vertex AI, or a custom gateway. Realm reports the free personal login fails here (gemini-cli 0.56.0, September 2026)

Realm's own notes on the protocol are in `docs/dev/acp-protocol.md`, and its client is `packages/adapters/src/acp/acp-adapter.ts`.

Step 6 — The Install button

The Install button runs the vendor's own install command, shown to the user word for word before it runs.

Tool	Install command
Claude Code	`npm install -g @anthropic-ai/claude-code`
Codex	`npm install -g @openai/codex`
Gemini CLI	`npm install -g @google/gemini-cli`
GitHub Copilot	`npm install -g @github/copilot`
OpenCode	`npm install -g opencode-ai`
Qwen Code	`npm install -g @qwen-code/qwen-code`
Grok	`npm install -g @xai-official/grok`
Cursor	`curl https://cursor.com/install -fsS | bash`
fx	`curl -fsSL https://fx.sh/setup.sh | bash`

Tool	Install command
Hermes	`curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash`
goose	`brew install block-goose-cli`
OpenHands	`uv tool install --python 3.12 openhands`

- **Store each route as data**, like `{ method: "npm", pkg: "@openai/codex" }`. Build both the text you show and the command you run from it, so the two can never differ.
- **npm, Homebrew and uv commands run directly, with no shell.** Only the `curl | bash` scripts go through `/bin/bash -c`, and there is no script install on Windows.
- **Check the installer exists first.** If npm is missing, say "Codex installs with npm, which comes with Node.js, and Node.js isn't on this Mac yet" instead of showing a raw error.
- **Run it safely:** input closed so a question fails fast, colour off (`NO_COLOR=1`, `FORCE_COLOR=0`), output streamed to the UI and capped at 256 KB, a 10-minute timeout, and one job per tool at a time.
- **Re-run Steps 1 and 2 when it finishes**, success or failure, skipping the cache, before telling the UI it's done.
- **Update the way it was installed.** Use the tool's own updater where one exists (`claude update`, `codex update`, `cursor-agent update`, `opencode upgrade`). Otherwise match the installer: a Homebrew copy gets `brew upgrade`, never npm, or the user ends up with two copies.
- **Find the latest version** from the npm registry (`registry.npmjs.org/<pkg>/latest`), Homebrew (`formulae.brew.sh/api/formula/<name>.json`) or PyPI (`pypi.org/pypi/<pkg>/json`). Cache it for six hours.

Step 7 — The Sign-in button

The Sign-in button runs the tool's own login command in a hidden terminal and passes the sign-in page and any code between that terminal and the user. Your app never sees a password or token.

1. **Offer it only where a login command exists:** `claude auth login`, `codex login`, `cursor-agent login`, `copilot login`, `opencode auth login`, `grok login`. Gemini and goose get an API-key field instead.

2. **Start the login in a pseudo-terminal** (`node-pty`, 100 by 30 characters), because these commands expect a real terminal. Run it in the user's home folder, not a project, where a repo's `.claude/settings.json` could steer the login.
3. **Let the CLI open the browser.** Read the sign-in link off its screen only to offer "Open the page again", so the user never gets two tabs.
4. **If the CLI asks for a code, show a paste box.** Type the code plus Enter into the terminal. Refuse control characters, because Ctrl-C in a terminal ends the login.
5. **Never log what was typed**, and cut it out of any error text you show.
6. **Give up after 10 minutes.** To cancel, send SIGHUP, then force-kill after 2 seconds.
7. **Confirm with a fresh Step 2 check.** A clean exit is not proof, so show "Signed in" only when the tool itself says yes.

Realm's first-run screen is just two of these buttons: "Sign in with Claude" and "Sign in with ChatGPT". Read Rules and risks before you ship the first one to other people.

Build order

Start with a one-file prototype that proves both subscriptions answer on your own Mac, then grow it into the app. Each stage ends in something you can run.

1. **Prototype.** One Node script finds `claude` and `codex`, prints their sign-in status, and sends one prompt through each. Done when both answer with no API key set.
2. **Adapter layer.** One interface every tool implements (below). Claude and Codex emit the same events: text, tool call, permission request, done, error. Done when no app code says "if Claude".
3. **ACP adapter.** One class runs every ACP tool from its table row. Done when adding a tool is one new row.
4. **Desktop shell.** An Electron app with the PATH fix in its main process and a Connections screen listing each tool's status.
5. **Install and Sign-in buttons**, following Steps 6 and 7.
6. **Your features on top.** The stock analysis work talks only to the adapter interface, and your own tools reach each session through an MCP server.
7. **Packaging.** A signed, notarized Mac build with auto-update. Windows comes later, once the `.cmd` spawning rules are tested.

```
interface AgentAdapter {
  kind: string;
  probe(): Promise<ProbeResult>;              // Steps 1 and 2
  start(opts: { cwd: string; model?: string; mcpServers?: McpServer[] }):
AgentHandle;
}
interface AgentHandle {
  events: AsyncIterable<SessionEvent>;        // text, tool,
permission_request, done, error
  send(text: string): Promise<void>;
  respondPermission(id: string, d: "allow" | "deny"): void;
  interrupt(): Promise<void>;
  dispose(): Promise<void>;
}
```

This is a trimmed version of Realm's own interface in `packages/adapters/src/types.ts`.

Rules and risks

Using your own subscription on your own Mac is the normal case. Offering "Sign in with Claude" to other people needs Anthropic's approval, so settle your Claude login plan before you publish.

- **Anthropic.** The Claude Agent SDK overview says third-party developers may not offer claude.ai login or rate limits in their products without prior approval, and should use API keys instead. For a public release, either ship "bring your own API key" for Claude or get approval first.
- **OpenAI.** I haven't checked OpenAI's terms for third-party apps driving `codex app-server` on a ChatGPT login. Check them before publishing too.
- **Google.** Plan on a Gemini API key, since Realm reports the free personal login no longer works for this.
- **Vendors change things.** Launch flags, status output and protocol shapes all moved during Realm's development. Keep every command in one table and add a test per tool that runs the real CLI.

Safety guards worth copying from Realm:

- Never read token files, and never log a code the user pasted.
- Show every install or update command before it runs, exactly as it will run.
- Never update a tool with a different installer than the one that put it there.

- Put a timeout on every check, install and sign-in.
- Kill every child process when the app quits. On Windows that takes `taskkill /T`, because the CLI runs under `cmd.exe`.

Open questions:

- [ ] Which Claude login plan for the public release: API key, Anthropic approval, or personal use only?
- [ ] Do OpenAI's terms allow a public app to drive Codex on a user's ChatGPT login?
- [ ] Which tools ship on day one? Claude Code and Codex alone would cover the subscription goal.
- [ ] Mac only at first, or Windows too?

What to copy from Realm

Realm is MIT licensed, so you can copy these files into your project as long as you keep its copyright and license text. Put them in a `THIRD-PARTY-NOTICES.md`.

Piece	Realm file
PATH fix	`apps/desktop/src/main/login-shell-path.ts`
Binary lookup and Windows spawning	`packages/adapters/src/spawn-target.ts`
Tool binary table	`apps/server/src/cli/bins.ts`
Claude detection and sign-in check	`packages/adapters/src/claude/probe.ts`
Claude settings-file login carry-over	`packages/adapters/src/claude/settings-auth.ts`
Claude adapter	`packages/adapters/src/claude/claude-adapter.ts`
Codex detection and sign-in check	`packages/adapters/src/codex/probe.ts`
Codex app-server connection	`packages/adapters/src/codex/connection.ts`, `codex-adapter.ts`
JSON-lines messaging	`packages/adapters/src/jsonrpc/stdio.ts`
ACP detection and model list	`packages/adapters/src/acp/probe.ts`
ACP adapter	`packages/adapters/src/acp/acp-adapter.ts`

Piece	Realm file
Install routes and update rules	`packages/contracts/src/cli.ts`
Install runner	`apps/server/src/cli/install.ts`
Tool status service	`apps/server/src/cli/service.ts`
Sign-in in a hidden terminal	`apps/server/src/agents/signin-service.ts`
Codex protocol notes	`docs/dev/codex-app-server-protocol.md`
ACP protocol notes	`docs/dev/acp-protocol.md`

These files import shared types from Realm's `@realm/contracts` package. Copy the types you need or swap in your own. Most of them have a test file beside them, which is worth copying too.